

```

#!/usr/bin/env python3
"""

Momentum vs Vanilla SGD Convergence Visualizer - Desktop Edition
A pure Python desktop application for comparing optimization algorithms.
Uses tkinter + matplotlib (numpy for computation).

Dependencies (all commonly pre-installed):
    numpy, matplotlib, tkinter (stdlib)
"""

import sys

# --- Defer matplotlib backend setup until GUI is actually needed ---
import numpy as np

# Tkinter is imported inside main() to avoid issues in headless environments

# =====
# OPTIMIZATION ALGORITHMS
# =====

def vanilla_sgd(grad_func, x0, lr=0.01, n_steps=100):
    """Vanilla Stochastic Gradient Descent:  $\theta_{t+1} = \theta_t - lr * grad$ """
    trajectory = [x0.copy()]
    x = x0.copy()
    for _ in range(n_steps):
        grad = grad_func(x)
        x = x - lr * grad
        trajectory.append(x.copy())
    return np.array(trajectory)

def momentum_sgd(grad_func, x0, lr=0.01, momentum=0.9, n_steps=100):
    """SGD with Momentum:  $v_t = \mu * v_{t-1} + lr * grad$ ;  $\theta_{t+1} = \theta_t - v_t$ """
    trajectory = [x0.copy()]
    x = x0.copy()
    v = np.zeros_like(x)
    for _ in range(n_steps):
        grad = grad_func(x)
        v = momentum * v + lr * grad
        x = x - v
        trajectory.append(x.copy())
    return np.array(trajectory)

# =====
# TEST FUNCTIONS
# =====

def bowl_func(x):
    """ $f(x,y) = x^2 + y^2$ """
    return x[0] ** 2 + x[1] ** 2

def bowl_grad(x):
    return np.array([2.0 * x[0], 2.0 * x[1]])

def rosenbrock_func(x, a=1.0, b=100.0):
    """ $f(x,y) = (a-x)^2 + b*(y-x^2)^2$ """
    return (a - x[0]) ** 2 + b * (x[1] - x[0] ** 2) ** 2

def rosenbrock_grad(x, a=1.0, b=100.0):
    dx = -2.0 * (a - x[0]) - 4.0 * b * x[0] * (x[1] - x[0] ** 2)
    dy = 2.0 * b * (x[1] - x[0] ** 2)
    return np.array([dx, dy])

def rastrigin_func(x, A=10.0):
    """Many local minima"""
    return (A * 2
            + (x[0] ** 2 - A * np.cos(2.0 * np.pi * x[0]))
            + (x[1] ** 2 - A * np.cos(2.0 * np.pi * x[1])))

```

```

def rastrigin_grad(x, A=10.0):
    dx = 2.0 * x[0] + 2.0 * np.pi * A * np.sin(2.0 * np.pi * x[0])
    dy = 2.0 * x[1] + 2.0 * np.pi * A * np.sin(2.0 * np.pi * x[1])
    return np.array([dx, dy])

def saddle_func(x):
    """f(x,y) = x^2 - y^2"""
    return x[0]**2 - x[1]**2

def saddle_grad(x):
    return np.array([2.0 * x[0], -2.0 * x[1]])

def elongated_func(x):
    """f(x,y) = 0.1*x^2 + y^2"""
    return 0.1 * x[0]**2 + x[1]**2

def elongated_grad(x):
    return np.array([0.2 * x[0], 2.0 * x[1]])

FUNCTIONS = {
    "Quadratic Bowl": {
        "func": bowl_func, "grad": bowl_grad,
        "min": np.array([0.0, 0.0]),
        "range": (-5, 5, -5, 5),
        "default_start": (-4.0, -4.0),
    },
    "Rosenbrock": {
        "func": rosenbrock_func, "grad": rosenbrock_grad,
        "min": np.array([1.0, 1.0]),
        "range": (-2, 2, -1, 3),
        "default_start": (-1.0, 2.0),
    },
    "Rastrigin": {
        "func": rastrigin_func, "grad": rastrigin_grad,
        "min": np.array([0.0, 0.0]),
        "range": (-5.12, 5.12, -5.12, 5.12),
        "default_start": (4.0, 4.0),
    },
    "Saddle Point": {
        "func": saddle_func, "grad": saddle_grad,
        "min": np.array([0.0, 0.0]),
        "range": (-5, 5, -5, 5),
        "default_start": (-4.0, -4.0),
    },
    "Elongated Bowl": {
        "func": elongated_func, "grad": elongated_grad,
        "min": np.array([0.0, 0.0]),
        "range": (-5, 5, -5, 5),
        "default_start": (-4.0, -4.0),
    },
}

# =====
# COLOR THEME
# =====

COLORS = {
    "bg_dark": "#1a1a2e",
    "bg_mid": "#16213e",
    "bg_light": "#0f3460",
    "panel": "#16213e",
    "text": "#e0e0e0",
    "text_dim": "#9e9e9e",
    "accent": "#4ECDC4",
    "vanilla": "#FF6B6B",
    "momentum": "#4ECDC4",
    "gold": "#FFD700",
    "surface_cmap": "viridis",
}

MPL_BG = "#1a1a2e"
MPL_AXES_BG = "#16213e"
MPL_TEXT = "#e0e0e0"

```

```

MPL_SPINE    = "#333355"
MPL_GRID     = "#2a2a4a"

# =====
#  MAIN APPLICATION CLASS (imports GUI libs here to keep module importable)
# =====

class SGDVisualizerApp:
    """Tkinter desktop application for SGD convergence visualization."""

    def __init__(self, root):
        import tkinter as tk
        from tkinter import ttk
        import matplotlib
        matplotlib.use("TkAgg")
        import matplotlib.pyplot as plt
        from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg, NavigationToolbar2Tk

        self.tk = tk
        self.ttk = ttk
        self.plt = plt
        self.FigureCanvasTkAgg = FigureCanvasTkAgg
        self.NavigationToolbar2Tk = NavigationToolbar2Tk

        self.root = root
        self.root.title("Momentum vs Vanilla SGD Convergence Visualizer")
        self.root.configure(bg=COLORS["bg_dark"])
        self.root.geometry("1440x920")
        self.root.minsize(1100, 750)

        self._setup_mpl_style()
        self._configure_styles()

        # --- state variables ---
        self.var_func    = tk.StringVar(value="Quadratic Bowl")
        self.var_lr      = tk.DoubleVar(value=0.01)
        self.var_momentum = tk.DoubleVar(value=0.90)
        self.var_iters   = tk.IntVar(value=100)
        self.var_start_x = tk.DoubleVar(value=-4.0)
        self.var_start_y = tk.DoubleVar(value=-4.0)

        self._build_ui()

        # Auto-run on first load
        self.root.after(200, self._run_optimization)

# -----
#  Matplotlib dark style
# -----
def _setup_mpl_style(self):
    plt = self.plt
    plt.rcParams.update({
        "figure.facecolor": MPL_BG,
        "axes.facecolor": MPL_AXES_BG,
        "axes.edgecolor": MPL_SPINE,
        "axes.labelcolor": MPL_TEXT,
        "xtick.color": MPL_TEXT,
        "ytick.color": MPL_TEXT,
        "text.color": MPL_TEXT,
        "grid.color": MPL_GRID,
        "grid.alpha": 0.6,
        "legend.facecolor": "#1f1f3a",
        "legend.edgecolor": MPL_SPINE,
        "font.family": "sans-serif",
        "font.size": 9,
    })

# -----
#  ttk styles
# -----
def _configure_styles(self):
    s = self.ttk.Style()
    s.theme_use("clam")
    bg = COLORS["bg_dark"]
    fg = COLORS["text"]
    accent = COLORS["accent"]

```

```

s.configure(".", background=bg, foreground=fg,
             fieldbackground="#222244", borderwidth=0,
             troughcolor="#222244", font=("Segoe UI", 10))
s.configure("TFrame", background=bg)
s.configure("TLabel", background=bg, foreground=fg, font=("Segoe UI", 10))
s.configure("Header.TLabel", font=("Segoe UI", 16, "bold"),
            foreground=accent, background=bg)
s.configure("Dim.TLabel", foreground=COLORS["text_dim"],
            background=bg, font=("Segoe UI", 9))
s.configure("TButton", background="#0f3460", foreground=fg,
            font=("Segoe UI", 10, "bold"), padding=(16, 8))
s.map("TButton", background=[("active", "#1a5276")])
s.configure("Accent.TButton", background=accent, foreground="#000000")
s.map("Accent.TButton", background=[("active", "#3dbdb5")])
s.configure("TLabelframe", background=bg, foreground=fg,
            relief="groove", bordercolor=MPL_SPINE)
s.configure("TLabelframe.Label", background=bg, foreground=accent,
            font=("Segoe UI", 10, "bold"))
s.configure("TScale", background=bg, troughcolor="#222244")
s.configure("TCombobox", fieldbackground="#222244", foreground=fg,
            selectbackground=accent, selectforeground="#000")
s.configure("TEntry", fieldbackground="#222244", foreground=fg)
s.configure("TSeparator", background=MPL_SPINE)
self.style = s

# -----
# Build entire UI
# -----
def _build_ui(self):
    tk = self.tk
    ttk = self.ttk

    # === HEADER ===
    hdr = ttk.Frame(self.root)
    hdr.pack(fill="x", padx=12, pady=(10, 4))

    ttk.Label(hdr, text="Momentum vs Vanilla SGD",
              style="Header.TLabel").pack(side="left")
    ttk.Label(hdr, text="Pure-Python Desktop Visualizer",
              style="Dim.TLabel").pack(side="left", padx=(14, 0))

    for label, color in [("Vanilla SGD", COLORS["vanilla"]),
                        ("Momentum SGD", COLORS["momentum"])]:
        frm = ttk.Frame(hdr)
        frm.pack(side="right", padx=8)
        c = tk.Canvas(frm, width=14, height=14,
                      bg=COLORS["bg_dark"], highlightthickness=0)
        c.create_oval(1, 1, 13, 13, fill=color, outline="")
        c.pack(side="left", padx=(0, 4))
        ttk.Label(frm, text=label, font=("Segoe UI", 9)).pack(side="left")

    ttk.Separator(self.root, orient="horizontal").pack(fill="x", padx=12, pady=4)

    # === CONTROLS ===
    ctrl = ttk.LabelFrame(self.root, text=" Parameters ", padding=8)
    ctrl.pack(fill="x", padx=12, pady=(0, 4))

    col = 0
    # Function selector
    ttk.Label(ctrl, text="Test Function").grid(
        row=0, column=col, sticky="w", padx=(0, 6))
    cb = ttk.Combobox(ctrl, textvariable=self.var_func,
                      values=list(FUNCTIONS.keys()),
                      state="readonly", width=18)
    cb.grid(row=1, column=col, padx=(0, 10), pady=(2, 0))
    cb.bind("<<ComboboxSelected>>", self._on_func_changed)

    # Learning rate
    col += 1
    self.lbl_lr = ttk.Label(ctrl, text="Learning Rate: 0.010")
    self.lbl_lr.grid(row=0, column=col, sticky="w", padx=(0, 6))
    ttk.Scale(ctrl, from_=0.001, to=0.5, variable=self.var_lr,
              orient="horizontal", length=150,
              command=self._on_lr_changed).grid(
        row=1, column=col, padx=(0, 10), pady=(2, 0))

```

```

# Momentum
col += 1
self.lbl_mom = ttk.Label(ctrl, text="Momentum: 0.90")
self.lbl_mom.grid(row=0, column=col, sticky="w", padx=(0, 6))
ttk.Scale(ctrl, from_=0.0, to=0.99, variable=self.var_momentum,
           orient="horizontal", length=150,
           command=self._on_mom_changed).grid(
    row=1, column=col, padx=(0, 10), pady=(2, 0))

# Iterations
col += 1
self.lbl_iter = ttk.Label(ctrl, text="Iterations: 100")
self.lbl_iter.grid(row=0, column=col, sticky="w", padx=(0, 6))
ttk.Scale(ctrl, from_=10, to=500, variable=self.var_iters,
           orient="horizontal", length=150,
           command=self._on_iter_changed).grid(
    row=1, column=col, padx=(0, 10), pady=(2, 0))

# Start position
col += 1
ttk.Label(ctrl, text="Start Position (x, y)").grid(
    row=0, column=col, sticky="w", padx=(0, 6))
sf = ttk.Frame(ctrl)
sf.grid(row=1, column=col, padx=(0, 10), pady=(2, 0))
ttk.Entry(sf, textvariable=self.var_start_x, width=7).pack(
    side="left", padx=(0, 4))
ttk.Entry(sf, textvariable=self.var_start_y, width=7).pack(
    side="left")

# Buttons
col += 1
ttk.Button(ctrl, text="\u25B6 Run Optimization",
            style="Accent.TButton",
            command=self._run_optimization).grid(
    row=0, column=col, rowspan=2, padx=(10, 0), sticky="ns")
ttk.Button(ctrl, text="Reset",
            command=self._reset).grid(
    row=0, column=col + 1, rowspan=2, padx=(6, 0), sticky="ns")
ctrl.columnconfigure(col, weight=1)

# === STATISTICS ROW ===
stat_frame = ttk.Frame(self.root)
stat_frame.pack(fill="x", padx=12, pady=(4, 4))

self.stat_labels = {}
for label, color_key in [("Vanilla SGD", "vanilla"),
                        ("Momentum SGD", "momentum")]:
    color = COLORS[color_key]
    box = tk.Frame(stat_frame, bg="#16213e", bd=0,
                   highlightthickness=1,
                   highlightbackground=color)
    box.pack(side="left", padx=(0, 8), fill="x", expand=True)
    tk.Label(box, text=label, bg="#16213e", fg=color,
             font=("Segoe UI", 11, "bold"),
             anchor="w").pack(fill="x", padx=8, pady=(6, 0))
    inner = tk.Frame(box, bg="#16213e")
    inner.pack(fill="x", padx=8, pady=(2, 6))
    self.stat_labels[color_key] = {}
    for stat_key, stat_name in [("loss", "Final Loss:"),
                                ("dist", "Distance to Min:")]:
        row_f = tk.Frame(inner, bg="#16213e")
        row_f.pack(fill="x")
        tk.Label(row_f, text=stat_name, bg="#16213e",
                 fg=COLORS["text_dim"],
                 font=("Segoe UI", 9)).pack(side="left")
        val_lbl = tk.Label(row_f, text="-", bg="#16213e",
                           fg="ffffff",
                           font=("Consolas", 10, "bold"))
        val_lbl.pack(side="right")
        self.stat_labels[color_key][stat_key] = val_lbl

# === MATPLOTLIB CANVAS (2x2) ===
canvas_frame = ttk.Frame(self.root)
canvas_frame.pack(fill="both", expand=True, padx=12, pady=(4, 10))

self.fig = self.plt.figure(figsize=(14, 6.5), dpi=100,
                             facecolor=MPL_BG)
self.fig.subplots_adjust(left=0.05, right=0.97, top=0.94

```

```

self.fig.subplots_adjust(left=0.05, right=0.97, top=0.94,
                        bottom=0.07, wspace=0.30, hspace=0.35)

gs = self.fig.add_gridspec(2, 2)
self.ax_3d = self.fig.add_subplot(gs[0, 0], projection="3d")
self.ax_contour = self.fig.add_subplot(gs[0, 1])
self.ax_converge = self.fig.add_subplot(gs[1, 0])
self.ax_velocity = self.fig.add_subplot(gs[1, 1])

self.canvas = self.figure.canvas
self.canvas.get_tk_widget().pack(fill="both", expand=True)

toolbar_frame = ttk.Frame(self.canvas)
toolbar_frame.pack(fill="x")
self.NavigationToolbar2Tk(self.canvas, toolbar_frame)

# -----
# Slider / Combo callbacks
# -----
def _on_lr_changed(self, val):
    self.lbl_lr.config(text=f"Learning Rate: {float(val):.3f}")

def _on_mom_changed(self, val):
    self.lbl_mom.config(text=f"Momentum: {float(val):.2f}")

def _on_iter_changed(self, val):
    self.lbl_iter.config(text=f"Iterations: {int(float(val))}")

def _on_func_changed(self, _event=None):
    info = FUNCTIONS[self.var_func.get()]
    self.var_start_x.set(info["default_start"][0])
    self.var_start_y.set(info["default_start"][1])

# -----
# Reset
# -----
def _reset(self):
    self.var_func.set("Quadratic Bowl")
    self.var_lr.set(0.01)
    self.var_momentum.set(0.90)
    self.var_iters.set(100)
    self.var_start_x.set(-4.0)
    self.var_start_y.set(-4.0)
    self.lbl_lr.config(text="Learning Rate: 0.010")
    self.lbl_mom.config(text="Momentum: 0.90")
    self.lbl_iter.config(text="Iterations: 100")
    for ck in self.stat_labels:
        for sk in self.stat_labels[ck]:
            self.stat_labels[ck][sk].config(text="-")
    self.ax_3d.cla()
    self.ax_contour.cla()
    self.ax_converge.cla()
    self.ax_velocity.cla()
    self.canvas.draw_idle()

# -----
# Run optimization + draw
# -----
def _run_optimization(self):
    name = self.var_func.get()
    info = FUNCTIONS[name]
    func = info["func"]
    grad = info["grad"]
    min_pt = info["min"]
    lr = self.var_lr.get()
    mom = self.var_momentum.get()
    n_steps = max(1, int(self.var_iters.get()))

    try:
        x0 = np.array([self.var_start_x.get(), self.var_start_y.get()])
    except Exception:
        x0 = np.array(info["default_start"], dtype=float)

    # --- run both optimizers ---
    traj_v = vanilla_sgd(grad, x0, lr=lr, n_steps=n_steps)
    traj_m = momentum_sgd(grad, x0, lr=lr, momentum=mom, n_steps=n_steps)

    self.update_stats(func, min_pt, traj_v, traj_m)

```

```

        self._plot_surface(name, info, traj_v, traj_m)
        self._plot_contour(name, info, traj_v, traj_m)
        self._plot_convergence(func, traj_v, traj_m)
        self._plot_velocity(traj_v, traj_m)
        self.canvas.draw_idle()

# -----
# Statistics
# -----
def _update_stats(self, func, min_pt, traj_v, traj_m):
    for ck, traj in [("vanilla", traj_v),
                     ("momentum", traj_m)]:
        f1 = float(func(traj[-1]))
        d = float(np.linalg.norm(traj[-1] - min_pt))
        self.stat_labels[ck]["loss"].config(
            text=f"{f1:.4e}" if f1 < 1e-3 else f"{f1:.4f}")
        self.stat_labels[ck]["dist"].config(
            text=f"{d:.4e}" if d < 1e-3 else f"{d:.4f}")

# -----
# 3D Surface
# -----
def _plot_surface(self, name, info, traj_v, traj_m):
    ax = self.ax_3d
    ax.cla()
    func = info["func"]
    xmin, xmax, ymin, ymax = info["range"]

    xa = np.linspace(xmin, xmax, 60)
    ya = np.linspace(ymin, ymax, 60)
    X, Y = np.meshgrid(xa, ya)
    Z = np.vectorize(lambda xi, yi: func(np.array([xi, yi])))(X, Y)

    ax.plot_surface(X, Y, Z, cmap=COLORS["surface_cmap"],
                    alpha=0.55, edgecolor="none",
                    antialiased=True, rstride=2, cstride=2)

    min_pt = info["min"]
    ax.scatter(*min_pt, func(min_pt), color=COLORS["gold"],
               s=90, marker="x", linewidths=2.5, zorder=10,
               label="Global Min")

    for traj, color, lbl in [
        (traj_v, COLORS["vanilla"], "Vanilla SGD"),
        (traj_m, COLORS["momentum"], "Momentum SGD"),
    ]:
        zv = np.array([func(p) for p in traj])
        ax.plot(traj[:, 0], traj[:, 1], zv,
                color=color, linewidth=2.2, label=lbl, alpha=0.9)
        # Start marker
        ax.scatter(traj[0, 0], traj[0, 1], zv[0],
                   color="lime", s=50, marker="D",
                   edgecolors="white", linewidths=0.6, zorder=8)
        # End marker
        ax.scatter(traj[-1, 0], traj[-1, 1], zv[-1],
                   color="red", s=50, marker="*",
                   edgecolors="white", linewidths=0.6, zorder=8)

    ax.set_title(f"{name} - 3D Loss Landscape",
                  fontsize=10, pad=6, color=MPL_TEXT)
    ax.set_xlabel("X", fontsize=8)
    ax.set_ylabel("Y", fontsize=8)
    ax.set_zlabel("Loss", fontsize=8)
    ax.tick_params(labels=7)
    ax.legend(loc="upper left", fontsize=7, framealpha=0.8)
    ax.view_init(elev=35, azim=-60)
    ax.set_facecolor(MPL_AXES_BG)
    ax.xaxis.pane.fill = False
    ax.yaxis.pane.fill = False
    ax.zaxis.pane.fill = False

# -----
# 2D Contour
# -----
def _plot_contour(self, name, info, traj_v, traj_m):
    ax = self.ax_contour
    ax.cla()

```

```

func = info["func"]
xmin, xmax, ymin, ymax = info["range"]

xa = np.linspace(xmin, xmax, 120)
ya = np.linspace(ymin, ymax, 120)
X, Y = np.meshgrid(xa, ya)
Z = np.vectorize(lambda xi, yi: func(np.array([xi, yi])))(X, Y)

zmin, zmax = Z.min(), Z.max()
if zmax > 0 and zmin >= 0:
    levels = (np.geomspace(max(zmin, 1e-3), zmax, 30)
               if zmax / max(zmin, 1e-10) > 100 else 30)
else:
    levels = 30

ax.contourf(X, Y, Z, levels=levels,
             cmap=COLORS["surface_cmap"], alpha=0.85)
ax.contour(X, Y, Z, levels=levels,
           colors="white", linewidths=0.3, alpha=0.35)

min_pt = info["min"]
ax.plot(min_pt[0], min_pt[1], marker="x",
        color=COLORS["gold"], markersize=14,
        markeredgewidth=2.5, zorder=10, label="Global Min")

for traj, color, lbl in [
    (traj_v, COLORS["vanilla"], "Vanilla SGD"),
    (traj_m, COLORS["momentum"], "Momentum SGD"),
]:
    ax.plot(traj[:, 0], traj[:, 1], color=color,
            linewidth=2, label=lbl, alpha=0.9)
    ax.plot(traj[0, 0], traj[0, 1], marker="D",
            color="lime", markersize=7,
            markeredgewidth=0.6, zorder=8)
    ax.plot(traj[-1, 0], traj[-1, 1], marker="*",
            color="red", markersize=9,
            markeredgewidth=0.6, zorder=8)

ax.set_title(f"{name} - 2D Contour View",
             fontsize=10, color=MPL_TEXT)
ax.set_xlabel("X", fontsize=8)
ax.set_ylabel("Y", fontsize=8)
ax.tick_params(labelsize=7)
ax.legend(loc="upper left", fontsize=7, framealpha=0.8)
ax.set_aspect("equal")
ax.set_facecolor(MPL_AXES_BG)

# -----
# Convergence (log scale)
# -----
def _plot_convergence(self, func, traj_v, traj_m):
    ax = self.ax_converge
    ax.cla()

    for traj, color, lbl in [
        (traj_v, COLORS["vanilla"], "Vanilla SGD"),
        (traj_m, COLORS["momentum"], "Momentum SGD"),
    ]:
        losses = np.maximum(
            np.array([func(p) for p in traj]), 1e-16)
        ax.semilogy(range(len(losses)), losses,
                    color=color, linewidth=2.2, label=lbl)

    ax.set_title("Convergence Comparison (Log Scale)",
                 fontsize=10, color=MPL_TEXT)
    ax.set_xlabel("Iteration", fontsize=8)
    ax.set_ylabel("Loss", fontsize=8)
    ax.tick_params(labelsize=7)
    ax.legend(loc="best", fontsize=7, framealpha=0.8)
    ax.grid(True, which="both", alpha=0.3)
    ax.set_facecolor(MPL_AXES_BG)

# -----
# Velocity / step-size
# -----
def _plot_velocity(self, traj_v, traj_m):
    ax = self.ax_velocity

```



```

ax.cla()

for traj, color, lbl in [
    (traj_v, COLORS["vanilla"], "Vanilla SGD"),
    (traj_m, COLORS["momentum"], "Momentum SGD"),
]:
    if len(traj) < 2:
        continue
    vel = np.linalg.norm(np.diff(traj, axis=0), axis=1)
    ax.plot(range(len(vel)), vel,
            color=color, linewidth=2.2, label=lbl)

ax.set_title("Step Size (Velocity Magnitude)",
             fontsize=10, color=MPL_TEXT)
ax.set_xlabel("Iteration", fontsize=8)
ax.set_ylabel("|| dx ||", fontsize=8)
ax.tick_params(labelsize=7)
ax.legend(loc="best", fontsize=7, framealpha=0.8)
ax.grid(True, alpha=0.3)
ax.set_facecolor(MPL_AXES_BG)

# =====
# ENTRY POINT
# =====

def main():
    import tkinter as tk
    import matplotlib
    matplotlib.use("TkAgg")
    import matplotlib.pyplot as plt

    root = tk.Tk()
    root.protocol("WM_DELETE_WINDOW",
                  lambda: (plt.close("all"), root.destroy()))
    SGDVisualizerApp(root)
    root.mainloop()

if __name__ == "__main__":
    main()

```